Sampling and Per-Precision Histograms with nixnan: An Experiment on `rd_nixnan.cu`

ganesh@cs.utah.edu
(written with Claude Code)

September 23, 2026

Abstract

This report documents a self-contained experiment (`Expts-9-23/`) demonstrating that `nixnan.so` helps understand a real GPU precision-sensitivity problem in three specific ways: (1) it can trade instrumentation overhead for observation completeness via the `SAMPLING` env var, (2) that trade-off can be driven through an arbitrary, externally-scripted schedule (here, an exponentially decaying rate) rather than a single fixed value, and (3) it reports, per floating-point format, exactly how much of that format's dynamic range a given run actually used. All numbers below come from real runs captured in `Logs/`; nothing is simulated or hand-typed.

Contents

1 The Example 1

2 "Straight nixnan": How These Runs Are Instrumented 2

3 Sampling, Level 1: Every Kernel Call 2

4 Sampling, Level 2: Once in 8 Kernel Calls 2

5 Sampling, Level 3: An Exponentially Decaying Schedule 3

6 Histograms per Precision Bucket 3

7 Conclusion: What nixnan Made Visible Here 4

1 The Example

`rd_nixnan.cu` solves a 1D reaction-diffusion PDE, $\partial u/\partial t = D\,\partial^2 u/\partial x^2 + \lambda u$, with explicit forward-Euler time stepping, on a 101-point grid for 1600 steps. The same step is implemented once per floating-point format — FP16, BF16, FP32, FP64 — as four separate CUDA kernels, so nixnan can attribute every exception and every exponent observation to a specific, known precision.

Why this example is precision-sensitive.

- The reaction term grows the solution by a fixed factor ($\sim$1.05$\times$) every step, so the exponent of every value climbs steadily regardless of format — the *only* thing that differs between

formats is how much exponent range each one has before it overflows. FP16's 5-bit exponent runs out first, at step 300; BF16's, FP32's, and FP64's 8/8/11-bit exponents all still have headroom left at step 1600 (see Section 6).

- Because the kernel is otherwise identical across formats, any behavioral difference observed — overflow or not, subnormal or not, NaN cascade or not — is attributable entirely to the format's exponent width, not to any change in the algorithm. That makes it a clean instrument for isolating precision effects, and correspondingly a clean instrument for showing what nixnan actually measures.

2 "Straight nixnan": How These Runs Are Instrumented

Every run in this experiment uses `nixnan.so` directly, with no wrapper script:

```
LD_PRELOAD=<path>/nixnan.so [ENV_VARS...] ./rd_nixnan
```

`nixnan.so` is an NVBit tool: preloaded into the CUDA process, it rewrites each kernel's SASS the first time that kernel is launched, inserting device-side checks after (and before) every floating-point instruction. At exit it prints an aggregate report, per format, of unique exception *sites* (distinct instruction/operand locations that ever fired) and *repeats* (additional occurrences beyond each site's first hit) — the distinction that matters throughout this report.

3 Sampling, Level 1: Every Kernel Call

The baseline (`SAMPLING` unset, which nixnan treats as "instrument everything"):

```
LD_PRELOAD=nixnan.so ./rd_nixnan
```

Result (`Logs/01_sampling_every_call.log`), FP16 — the only format that overflows in this build:

```
NaN: 22 (60534 repeats)
Infinity: 22 (639 repeats)
```

This is the reference point every other sampling level in this report is compared against.

4 Sampling, Level 2: Once in 8 Kernel Calls

```
SAMPLING=8 LD_PRELOAD=nixnan.so ./rd_nixnan
```

Result (`Logs/02_sampling_every_8.log`):

```
NaN: 22 (7524 repeats)
Infinity: 17 (66 repeats)
```

Repeats drop by almost exactly 8× (60534/7524 ≈ 8.05), consistent with observing 1/8 of the kernel launches. But note the unique Infinity-site count also dropped, 22 → 17 — five distinct exception sites simply never got observed at this sampling rate. This is the first concrete evidence that `SAMPLING` trades completeness for overhead, not just verbosity for overhead.

A regression, now fixed. The pre-existing `run_nixnan_experiment.sh` in this directory carries a comment, from an earlier investigation, stating *"SAMPLING is a no-op in this nixnan build."* We re-tested that claim directly against the current build before writing this experiment (comparing `SAMPLING=1` against `SAMPLING=100000` on `reaction_diffusion_gpu_fp32`): at `SAMPLING=100000` every exception count dropped to exactly zero, which only happens if `SAMPLING` is genuinely gating instrumentation. `SAMPLING` is no longer a no-op in the `auto-nixnan` branch as built here — whatever caused the earlier no-op behavior has since been fixed.

5 Sampling, Level 3: An Exponentially Decaying Schedule

`nixnan.so` reads `SAMPLING` once at process start; there is no way to vary it mid-run. A decaying/backoff sampling rate therefore has to be built at the orchestration level, as a sequence of short runs ("snaps"), each holding `SAMPLING` constant. `shell-scripts/sampling_schedule.json` specifies that sequence (full rationale in its own header comment):

```
{
  "snaps_per_level": 4,
  "levels": [1, 2, 4, 8],
  "cycles": 2
}
```

i.e. step 4 times at spacing 1, double to spacing 2 and step 4 more times, double again to 4, double again to 8, then reset back to spacing 1 for a second cycle. `shell-scripts/03_sampling_exp_decay.sh` runs this schedule and tabulates the FP16 counts per snap (`Logs/03_sampling_exp_decay_summary.tsv`):

SAMPLING level	NaN unique	NaN repeats	Inf unique	Inf repeats
1	22	60534	22	639
2	22	30246	21	312
4	22	15103	17	154
8	22	7524	17	66

Every one of the 4 snaps at a given level is byte-identical (the program is deterministic and each snap is a fresh, independent process), and **cycle 2 is byte-identical to cycle 1 at every level** — the reset back to spacing 1 reproduces the exact baseline from Section 3, confirming there is no hidden cross-run state. See Figure 1 (`viz/sampling_decay.svg`).

6 Histograms per Precision Bucket

`HISTOGRAM=1` enables exponent-histogram tracking for every instrumented FP instruction, independent of the exception checker; at exit nixnan prints the [min, max] exponent actually observed for each format that had any instrumented activity:

```
HISTOGRAM=1 LD_PRELOAD=nixnan.so ./rd_nixnan
```

Result (`Logs/04_histograms_per_precision.log`):

```
Exponent range for bf16: [zero, 113]
Exponent range for f16: [zero, inf]
Exponent range for f32: [zero, 113]
Exponent range for f64: [zero, 113]
```

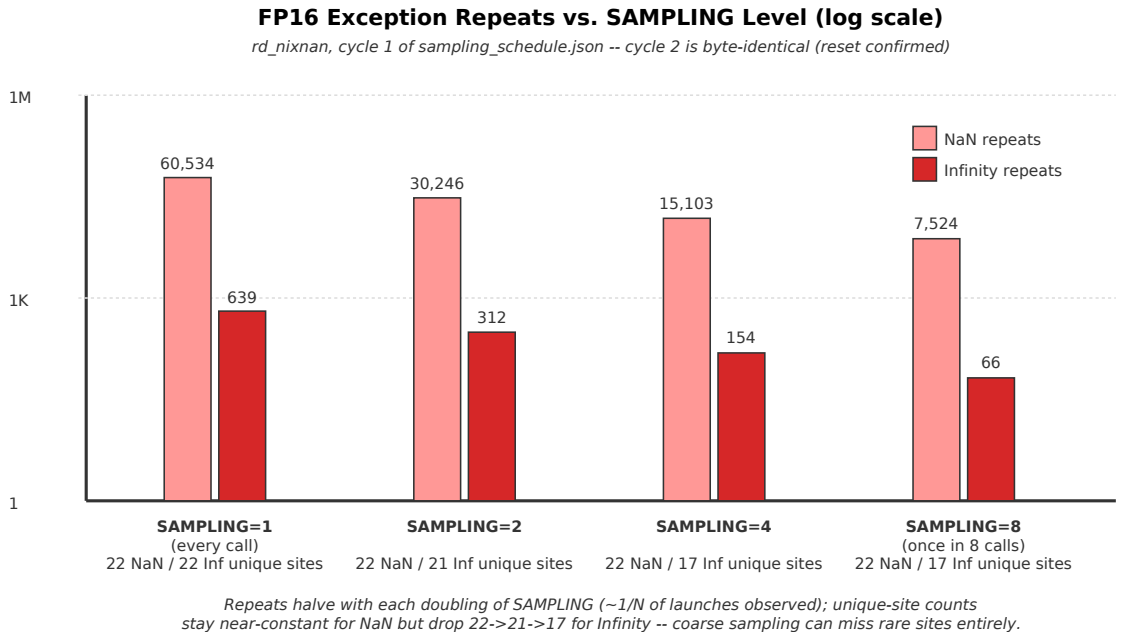


Figure 1: Repeats halve with each doubling of SAMPLING; unique-site counts stay near-constant for NaN but step down for Infinity.

This is the direct, per-format confirmation of the precision-sensitivity claim from Section 1: FP16 is the only format whose exponent range is exhausted (it reports `inf` instead of a finite maximum) within 1600 steps. BF16, FP32, and FP64 all reach the identical exponent 113 — they are following the exact same growth trajectory (same algorithm, same input) and have not yet diverged from each other; only their exponent *ceilings* differ, and only FP16's has been reached.

7 Conclusion: What nixnan Made Visible Here

Across four short experiments on one simple, well-understood kernel, `nixnan` surfaced three things that would not be visible from the program's own output (a plain run of `rd_nixnan` only prints `max=...` per step, with no notion of exception sites, repeats, or exponent range):

1. **Sampling is a real overhead/completeness knob, not a free lunch.** Coarser sampling doesn't just reduce reported repeats proportionally — it can silently drop entire exception sites from the report (22 → 17 unique Infinity sites, Section 4), which matters if the goal is finding every distinct bug location rather than just gauging overall severity.
2. **A fixed env var can still drive an arbitrary sampling schedule**, by scripting a sequence of short, independent runs around it (Section 5) — and because `rd_nixnan` is fully deterministic, that schedule is provably reproducible: cycle 2 reproduces cycle 1 exactly.
3. **Per-format exponent histograms directly confirm *which* format is precision-sensitive and by how much** (Section 6), turning "FP16 fails and the others probably don't" into a specific, quantified claim: FP16 alone hits `inf`; the other three all reach exactly the same exponent, 113, with headroom still remaining to their much larger ceilings.

As a side effect, re-verifying `SAMPLING` against the current build (Section 4) also caught that a

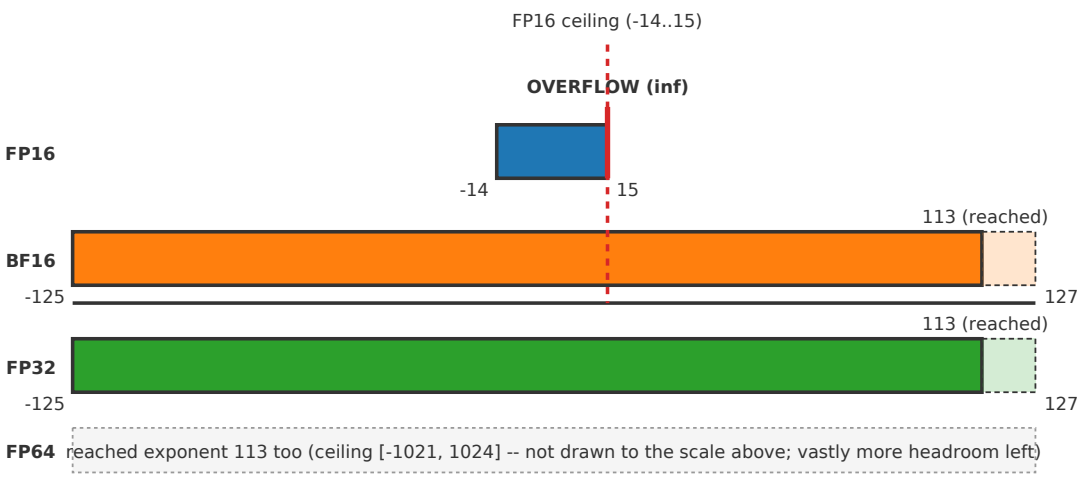


Figure 2: Exponent range actually reached by each format through step 1600. FP16's fixed capacity is $[-14, 15]$ and it exceeds it (overflow); BF16/FP32/FP64 all reach the same exponent, 113, with room remaining to their much wider ceilings.

previously-documented no-op regression in `nixnan.so` has since been fixed — itself an example of nixnan-adjacent tooling needing the same kind of empirical re-verification its own instrumentation provides for user code.